Name : Adhithya.S
Reg. No: 192412352

---

CODE:

```
import numpy as np

# XOR Training Dataset
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=float)

# Expected Output
y = np.array([
    [0],
    [1],
    [1],
    [0]
], dtype=float)


# Sigmoid Activation Function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))


# Derivative of Sigmoid Function
def sigmoid_derivative(x):
    return x * (1 - x)


# Set random seed for reproducible results
np.random.seed(42)

# Network Architecture
input_neurons = 2
hidden_neurons = 4
output_neurons = 1

# Initialize Weights and Biases
weights_input_hidden = np.random.uniform(
    -1, 1, (input_neurons, hidden_neurons)
)

bias_hidden = np.random.uniform(
    -1, 1, (1, hidden_neurons)
)

weights_hidden_output = np.random.uniform(
    -1, 1, (hidden_neurons, output_neurons)
)

bias_output = np.random.uniform(
    -1, 1, (1, output_neurons)
)


# Training Parameters
learning_rate = 0.5
```

```
epochs = 10000


# Backpropagation Training
for epoch in range(epochs):

    # ---------- FORWARD PROPAGATION ----------

    # Input to Hidden Layer
    hidden_input = np.dot(X, weights_input_hidden) + bias_hidden

    # Hidden Layer Output
    hidden_output = sigmoid(hidden_input)

    # Hidden to Output Layer
    output_input = np.dot(
        hidden_output,
        weights_hidden_output
    ) + bias_output

    # Final Network Output
    predicted_output = sigmoid(output_input)


    # ---------- ERROR CALCULATION ----------

    error = y - predicted_output


    # ---------- BACKPROPAGATION ----------

    # Output Layer Delta
    output_delta = (
        error * sigmoid_derivative(predicted_output)
    )

    # Hidden Layer Error
    hidden_error = np.dot(
        output_delta,
        weights_hidden_output.T
    )

    # Hidden Layer Delta
    hidden_delta = (
        hidden_error * sigmoid_derivative(hidden_output)
    )


    # ---------- UPDATE WEIGHTS ----------

    weights_hidden_output += (
        learning_rate *
        np.dot(hidden_output.T, output_delta)
    )

    bias_output += (
        learning_rate *
        np.sum(output_delta, axis=0, keepdims=True)
    )

    weights_input_hidden += (
        learning_rate *
        np.dot(X.T, hidden_delta)
    )
```

```
    bias_hidden += (
        learning_rate *
        np.sum(hidden_delta, axis=0, keepdims=True)
    )


    # Display error every 1000 epochs
    if epoch % 1000 == 0:
        mse = np.mean(error ** 2)
        print(
            "Epoch:",
            epoch,
            "MSE:",
            round(mse, 6)
        )


# ---------- TESTING ----------

print("\nTesting the Trained Neural Network")
print("-" * 40)

# Perform final forward propagation
hidden_input = np.dot(
    X,
    weights_input_hidden
) + bias_hidden

hidden_output = sigmoid(hidden_input)

output_input = np.dot(
    hidden_output,
    weights_hidden_output
) + bias_output

predicted_output = sigmoid(output_input)


# Display Results
for i in range(len(X)):

    predicted_class = (
        1 if predicted_output[i][0] >= 0.5 else 0
    )

    print(
        "Input:",
        X[i],
        "Expected:",
        int(y[i][0]),
        "Predicted:",
        predicted_class,
        "Output:",
        round(predicted_output[i][0], 4)
    )
```

OUTPUT :

```
Epoch: 0 MSE: 0.315398
Epoch: 1000 MSE: 0.009001
Epoch: 2000 MSE: 0.002133
Epoch: 3000 MSE: 0.001161
Epoch: 4000 MSE: 0.000789
Epoch: 5000 MSE: 0.000594
Epoch: 6000 MSE: 0.000475
Epoch: 7000 MSE: 0.000396
Epoch: 8000 MSE: 0.000338
Epoch: 9000 MSE: 0.000295

Testing the Trained Neural Network
----------------------------------------
Input: [0. 0.] Expected: 0 Predicted: 0 Output: 0.0138
Input: [0. 1.] Expected: 1 Predicted: 1 Output: 0.9827
Input: [1. 0.] Expected: 1 Predicted: 1 Output: 0.9842
Input: [1. 1.] Expected: 0 Predicted: 0 Output: 0.0175
```